

```

ioctl(fd, MEMGETINFO, &mtd_info);

printf("MTD type: %u\n", mtd_info.type);
printf("MTD total size : %u bytes\n", mtd_info.size);
printf("MTD erase size : %u bytes\n", mtd_info.erasize);

return 0;
}

```

Error handling has been omitted for brevity. The *mtd_info_t* structure is used with the *MEMGETINFO* command. The MTD type can be *MTD_ABSENT*, *MTD_RAM*, *MTD_ROM*, *MTD_NAND*, *MTD_NOR*, etc, which are defined in the *mtd/mtd-abi.h* header file. The *mtd_info.size* indicates the size of the whole device (i.e., the partition, in this case). Finally, *mtd_info.erasize* indicates the sector size. During an erase operation, this is the minimum size that can be erased, as we'll see later.

Reading MTD devices is similar to ordinary devices:

```

/* read something from last sector */
unsigned char buf[64];
lseek(fd, -mtd_info.erasize, SEEK_END);
read(fd, buf, sizeof(buf));

```

A write operation can be performed in the same way, provided the sector has been erased previously. Finally, we come to the erase operation. Here is an example of erasing a partition, sector by sector:

```

void erase_partition(mtd_info_t *mtd_info, int fd) {
    erase_info_t ei;
    ei.length = mtd_info->erasize;

    for(ei.start = 0; ei.start < mtd_info->size; ei.start += mtd_info->erasize) {
        ioctl(fd, MEMUNLOCK, &ei);
        ioctl(fd, MEMERASE, &ei);
    }
}

```

All sectors of the device are writeable after this erase operation. Notice the use of *MEMUNLOCK* before *MEMERASE*, which is essential to allow the erase operation.

Accessing MTDs from kernel space

This section will repeat the functions explained in the previous section, but in kernel space. This needs a separate section since the erase operation is more complex here — the erase operation may sleep and therefore the kernel programmer has to wait until the operation is completed. This is the case for applications too, but the sleep is transparently taken care of by the scheduler.

As explained earlier, the first MTD information is the *mtd_info* structure. This is retrieved by iterating through all registered MTD devices:

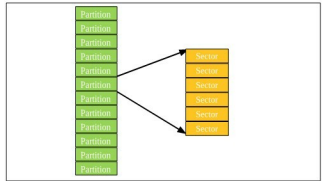


Figure 1: An MTD device

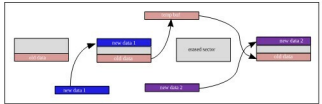


Figure 2: An MTD write operation

```

#include <linux/kernel.h>
#include <linux/mtd/mtd.h>
#include <linux/err.h>

static struct mtd_info *mtd_info = NULL;

int init_module(void) {
    int num;
    for(num = 0; num < 64; num++) {
        mtd_info = get_mtd_device(NULL, num);
        if(IS_ERR(mtd_info)) {
            printk("No device for num %d\n", num);
            continue;
        }
        if(mtd_info->type == MTD_ABSENT) {
            put_mtd_device(mtd_info);
            continue;
        }
        if(strcmp(mtd_info->name, "userdata")) {
            put_mtd_device(mtd_info);
            continue;
        }
        printk("MTD type: %u\n", mtd_info->type);
        printk("MTD total size : %u bytes\n", mtd_info->size);
        printk("MTD erase size : %u bytes\n", mtd_info->erasize);
        return 0;
    }
    mtd_info = NULL;
    return 0;
}

void cleanup_module(void){

```